

Regressione Logistica (LogR)

Modello

La regressione logistica è un modello utilizzato per problemi di **classificazione binaria**, dove la variabile di risposta è dicotomica (es. "Sì/No", "Malato/Sano", "Default/Non-default").

Il modello combina una **combinazione lineare di feature** con una **funzione logistica (sigmoide)** per mantenere l'output tra 0 e 1 (probabilità):

$$\sigma(z) = \frac{1}{1 + \exp(-z)}$$

Nel caso specifico, la probabilità che l'istanza appartenga alla classe positiva ($y = 1$) è:

$$P(y^{(i)} = 1 | x^{(i)}) = \frac{1}{1 + \exp(-(\beta_0 + \sum_{j=1}^p \beta_j x_j^{(i)}))}$$

La probabilità della classe negativa è il complemento: $P(y^{(i)} = 0) = 1 - P(y^{(i)} = 1)$.

In generale, la probabilità congiunta è descritta da una distribuzione di Bernoulli:

$$P(y^{(i)}, x^{(i)}) = \{P(y^{(i)} = 1)^{y^{(i)}} \cdot (1 - P(y^{(i)} = 1))^{1-y^{(i)}}\} \text{ Distribuzione Bernoulli}$$

Stima dei Coefficienti

A differenza della regressione lineare che minimizza l'errore quadratico, la regressione logistica **massimizza la likelihood** (probabilità osservare i dati dati i parametri):

$$L(\beta; y, X) = \prod_{i=1}^n P(y^{(i)} = 1)^{y^{(i)}} \cdot (1 - P(y^{(i)} = 1))^{1-y^{(i)}}$$

In pratica si massimizza il **log-likelihood** per stabilità numerica:

$$\ell(\beta) = \sum_{i=1}^n [y^{(i)} \log(P(y^{(i)} = 1)) + (1 - y^{(i)}) \log(1 - P(y^{(i)} = 1))]$$

La massimizzazione avviene tramite **ottimizzazione iterativa** (Gradient Descent, Newton-Raphson, IRLS) poiché non esiste una soluzione analitica chiusa come nella regressione lineare.

Complessità Computazionale

Training

- **Complessità principale:** $O(p \cdot k \cdot n)$
 - k = numero di iterazioni dell'algoritmo di ottimizzazione (solitamente 20-100)
 - p = numero di feature
 - n = numero di osservazioni
 - Dipende fortemente dal metodo di ottimizzazione scelto
- **Memoria:** $O(p)$ per il vettore dei coefficienti + $O(n)$ per il batch

Inference

- **Complessità:** $O(p)$ per istanza (una moltiplicazione vettore + funzione sigmoid)
- **Memoria:** $O(p)$ per il vettore dei coefficienti

Considerazioni sulla Scalabilità

La **dipendenza iterativa** dalla convergenza rende la regressione logistica **meno prevedibile** della regressione lineare:

- Su dataset con feature altamente correlate, la convergenza può essere lenta (molte iterazioni)
- La separazione completa dei dati causa **divergenza numerica** (coefficienti $\rightarrow \pm\infty$)
- Per dataset **very large** ($n \gg 1M$), il training rimane trattabile grazie alla linearità della loss

Vantaggio rispetto a modelli come SVM o Neural Networks: rimane computazionalmente efficiente anche su dataset di media grandezza senza acceleratori GPU.

Rappresentazione Interna

Come la regressione lineare, il modello rappresenta internamente il modello come un **vettore di pesi** $\beta = [\beta_0, \beta_1, \dots, \beta_p]$.

Differenze rispetto a LR:

1. I coefficienti **non hanno interpretazione additiva diretta** (non è "aumenta di β_j "), ma **moltiplicativa** (vedi sezione Vincoli)
2. La **trasformazione sigmoide** non è invertibile facilmente, quindi l'effetto di una feature sulla probabilità **dipende dal valore attuale** di tutte le altre feature (non è localmente lineare come in LR)

Implicazioni per la spiegabilità:

- Rappresentazione **ancora compatta e relativamente trasparente**
- L'interpretazione è però **meno intuitiva** di LR: i pesi non corrispondono direttamente a variazioni di probabilità
- La previsione è ancora completamente tracciabile (non è una scatola nera), ma il ragionamento richiede comprensione della trasformazione logistica
- Per istanze vicino a probabilità 0.5, piccole variazioni nei pesi causano grandi variazioni nella previsione (derivata della sigmoid è massima a 0.5)

Vincoli sui Dati

Interpretazione Moltiplicativa dei Pesi

Dalla trasformazione della funzione logistica, gli **odds** (rapporto di probabilità) seguono una relazione **lineare sui log-odds**:

$$\text{odds} = \frac{P(y=1)}{P(y=0)} = \exp\left(\beta_0 + \sum_{j=1}^p \beta_j x_j\right)$$

Aumentando di 1 unità una feature j , l'effetto moltiplicativo sugli odds è:

$$\frac{\text{odds}_{x_j+1}}{\text{odds}} = \exp(\beta_j)$$

Interpretazione pratica: Se $\beta_j = 0.5$, aumentare x_j di 1 unità **moltiplica gli odds per $e^{0.5} \approx 1.65$** , cioè aumenta del 65%. (Non aggiunge 0.5 come in LR!)

Assunzioni Fondamentali

1. **Linearità del log-odds:** $\log(\text{odds}) = \beta_0 + \sum_j \beta_j x_j$. La relazione è lineare nello **log-spazio degli odds**, non nello spazio della probabilità. Relazioni non lineari rimangono non catturate

2. **Linearità nei vincoli:** come LR, le feature si combinano linearmente nel log-odds. Interazioni devono essere aggiunte manualmente
3. **Separazione completa:** se una feature separa perfettamente le due classi (tutti i valori di una classe da un lato, tutti dell'altra dall'altro), il modello non può identificare un peso finito. L'algoritmo **diverge** (coefficiente $\rightarrow +\infty$ o $-\infty$)
4. **Distribuzione binomiale:** la variabile di risposta deve seguire una **distribuzione di Bernoulli** (ogni osservazione è indipendente e ha probabilità fissa)
5. **Indipendenza delle misure:** le osservazioni non devono essere correlate (no dati temporali consecutivi senza controllo)
6. **Feature fisse:** le feature devono essere considerate costanti, senza errore di misurazione
7. **Assenza di multicollinearità:** feature fortemente correlate causano instabilità dei coefficienti (come in LR, ma peggio perché l'ottimizzazione iterativa può oscillare)

Nota importante: L'omoschedasticità **non è un vincolo** come in LR, poiché i valori della risposta possono assumere solo 0 o 1 (non una distribuzione continua)

Capacità Predittive

Punti di Forza

- **Output probabilistico:** diversamente da un classificatore duro, fornisce una stima di confidenza della previsione
- **Efficienza computazionale:** training veloce anche su dataset moderatamente grandi
- **Interpretabilità:** i pesi, sebbene moltiplicativi, rimangono interpretabili
- **Linearità decisionale:** il confine decisionale è una retta (in 2D) o iperpiano (in p-D)

Punti di Debolezza

- **Non cattura non linearità:** come LR, pattern non lineari complessi rimangono invisibili al modello
- **Sensibile a separazione completa:** diverge numericamente se esiste una feature "perfetta"
- **Interazioni nascoste:** effetti di feature che dipendono l'uno dall'altro rimangono invisibili

Metriche per la Confidenza

Metriche Pure

Confusion Matrix (Matrice di Confusione)

Confronta le classi reali con quelle predette, generando 4 categorie:

- **TP (True Positives):** previsione positiva corretta
- **TN (True Negatives):** previsione negativa corretta
- **FP (False Positives):** errore di tipo I, previsione positiva errata
- **FN (False Negatives):** errore di tipo II, previsione negativa errata

Accuracy

Frazione di previsioni corrette:

$$ACC = \frac{TP + TN}{TP + TN + FP + FN}$$

Limite: metrica misleading se le classi sono sbilanciate. Un classificatore che sempre predice la classe maggioritaria avrà accuracy alta

Sensitivity (Recall / True Positive Rate)

Frazione di positivi correttamente identificati:

$$Sens = \frac{TP}{TP + FN}$$

Risponde: "Su tutti i veri positivi, quanti abbiamo identificato?"

Specificity (True Negative Rate)

Frazione di negativi correttamente identificati:

$$Spec = \frac{TN}{TN + FP}$$

Risponde: "Su tutti i veri negativi, quanti abbiamo identificato?"

Precision

Frazione di previsioni positive corrette:

$$PREC = \frac{TP}{TP + FP}$$

Risponde: "Tra tutte le istanze che abbiamo predetto come positive, quante sono veramente positive?"

Recall

Alias per Sensitivity:

$$REC = \frac{TP}{TP + FN}$$

F1-Score

Media armonica tra Precision e Recall, utile quando si vuole bilanciare i due:

$$F1 = 2 \frac{PREC \cdot REC}{PREC + REC}$$

Quando usarlo: quando le classi sono sbilanciate e sia falsi positivi che falsi negativi hanno costo significativo

ROC Curve e AUC

ROC Curve: rappresentazione grafica del trade-off tra True Positive Rate (Sensitivity) sull'asse y e False Positive Rate (1 - Specificity) sull'asse x, al variare del threshold di classificazione.

- **Modello ideale:** curva passa per il punto (0,1) in alto a sinistra
- **Modello casuale:** curva segue la diagonale (AUC = 0.5)

AUC (Area Under the Curve): area sotto la ROC curve, quantifica la capacità globale del modello di discriminare tra le classi

- **AUC = 1.0:** discriminazione perfetta
- **AUC = 0.5:** modello casuale
- **AUC < 0.5:** peggior del caso

AUC è **invariante al threshold**, quindi è preferibile ad Accuracy per valutazioni comparative.

Z-Statistic e p-value

Analogo della t-statistic in LR, misura la significatività statistica di ogni coefficiente:

$$Z = \frac{\beta_j}{SE(\beta_j)}$$

Dove $SE(\beta_j)$ è l'errore standard del coefficiente.

- **Valori assoluti grandi di Z** → forte evidenza che il coefficiente è significativamente diverso da 0
- **p-value associato** → probabilità di osservare Z così estremo sotto l'ipotesi nulla ($\beta_j = 0$)
- **Convention:** $p < 0.05$ indica significatività

Metriche per la Comprensione e Spiegabilità

Effect Plot

Simile a LR, rappresenta l'effetto di una feature sulla **probabilità predetta** (non sugli odds, che è più difficile da interpretare).

Per ogni valore della feature, calcolare:

$$P(y = 1 \mid x_j = v, x_{\text{other}} = \text{media})$$

Il grafico mostra come la probabilità predetta varia col valore della feature, mantenendo le altre feature ai loro valori medi.

Vantaggio rispetto a LR: la trasformazione sigmoide rende visibile se l'effetto è principalmente presso certi valori della feature (grafico non è una retta, è una curva).

Weight Plot (Coefficienti)

Analogo a LR, rappresenta graficamente i coefficienti β_j ordinati per valore assoluto.

Caveat: il valore di β_j non corrisponde direttamente a una variazione di probabilità (è moltiplicativo sugli odds, non additivo sulla probabilità). Quindi il grafico ha **minore valore interpretativo** che in LR.

Odds Ratio

Espressione diretta dell'impatto di una feature sugli odds:

$$OR_j = \exp(\beta_j)$$

Interpretazione: "Aumentare feature j di 1 unità moltiplica gli odds per OR_j "

Esempio: se $\beta_j = 0.5$ e $OR_j = 1.65$, aumentare la feature del 65% gli odds di appartenenza alla classe positiva.

Limiti di Predizione

Non Linearità

Come LR, il modello non cattura pattern non lineari. La sigmoide è una trasformazione "accessoria" che non supera questo limite.

Separazione Completa

Se una feature separa perfettamente le classi, l'algoritmo di ottimizzazione **diverge numericamente**: il coefficiente tende a $+\infty$ o $-\infty$. Soluzioni:

- Penalizzare i pesi (Ridge o Lasso logistica)

- Rimuovere la feature (ma significa perdere informazione)
- Usare Firth's bias-reduced logistic regression (metodo specializzato)

Overfitting

Con un numero elevato di feature rispetto al numero di osservazioni ($p \gg n$), il modello può facilmente sovradattarsi al training set. Ridge o Lasso logistica riducono il rischio.

Sensibilità al Classe Sbilanciate

Il modello di default è addestrato minimizzando il likelihood globale. Se una classe è molto rara (es. 1% positivi, 99% negativi), il modello tenderà a predire sempre la classe maggioritaria. Soluzioni:

- Usare pesi di classe inversi alla frequenza (class weights)
- Ricampionare (oversampling della classe rara o undersampling della classe maggioritaria)
- Regolare il threshold di classificazione (non usare 0.5 di default)

Limiti di Spiegabilità

Interpretazione Moltiplicativa Complessa

A differenza di LR dove un coefficiente corrisponde a una variazione additiva, qui $\exp(\beta_j)$ è moltiplicativo e non intuitivo per la maggior parte degli utenti. Una feature che aumenta gli odds del 65% non è semplice da comunicare rispetto a "la previsione aumenta di 10 unità".

Dipendenza dal Contesto

L'effetto di una feature sulla probabilità predetta **dipende dai valori di tutte le altre feature**. Per una feature, l'aumento della probabilità è maggiore quando le altre feature sono tali che la probabilità predetta è intorno a 0.5 (dove la sigmoid è più ripida). Questo rende difficile fornire una spiegazione "universale" dell'effetto di una feature.

Piccoli Coefficienti Difficili da Valutare

Se β_j è piccolo (es. 0.01), l'effetto $\exp(0.01) \approx 1.01$ (aumento del 1%) è difficile da discernere dalla variabilità naturale. Questo crea incertezza sull'importanza reale della feature.

Interazioni Nascoste

Se due feature hanno un effetto congiunto non additivo, il modello base non lo cattura. Le interazioni devono essere introdotte manualmente, il che aggiunge complessità nel comunicare i risultati.

Confronto con altri Algoritmi

Vs. Regressione Lineare su Risposta 0/1

- **LR su dati binari:** produce previsioni fuori da $[0,1]$, ha issue di interpretazione, non è concepita per classificazione
- **Logistica:** mantiene output in $[0,1]$, fornisce probabilità direttamente interpretabili
- **Conclusione:** Logistica è sempre preferibile per problemi di classificazione binaria

Vs. Linear Discriminant Analysis (LDA)

Entrambi creano confini decisionali **lineari**. Tuttavia:

- **Logistica:** modella direttamente $P(y = 1 | x)$ (approccio discriminativo)
- **LDA:** modella la distribuzione congiunta di x e y (approccio generativo), assumendo normalità multivariata di x per ogni classe

Comparazione empirica: spesso producono risultati simili, ma logistica è **preferibile se:**

- Le assunzioni gaussiane sono violate (dati non normali)
- Le classi hanno covarianze molto diverse

Vs. Logistic Regression Multinomiale

- **Binaria:** classifica in due classi
- **Multinomiale:** estende a $K > 2$ classi
- Usa l'approccio **softmax**: per ogni classe k , stima $P(y = k)$ come funzione softmax dei pesi specifici di quella classe

Vs. Lasso/Ridge per Logistica

Come per LR:

- **Logistica pura:** minimizza log-likelihood senza penalità
- **Ridge logistica:** aggiunge penalità L2, riduce coefficienti, mantiene tutte le feature
- **Lasso logistica:** aggiunge penalità L1, azzeramento selettivo di coefficienti (feature selection)
- **Quando usare:** multicollinearità o separazione completa $\rightarrow$ Ridge/Lasso

Vs. Generalized Additive Models (GAM) Logistici

- **Logistica base:** log-odds lineari $\log(\text{odds}) = \beta_0 + \sum \beta_j x_j$
- **GAM logistica:** log-odds additivi ma non lineari $\log(\text{odds}) = \beta_0 + \sum f_j(x_j)$ dove f_j sono funzioni non lineari (es. splines)
- **Vantaggio:** preserva linearità nel log-odds (additività) ma cattura non linearità
- **Svantaggio:** la rappresentazione interna è più complessa (curve invece di rette)

Vs. Support Vector Machines (SVM)

- **Logistica:** output probabilistico, interpretazione diretta
- **SVM:** output hard (classe) o soft (margine), è un classificatore geometrico
- **Trade-off:** SVM ha spesso migliore capacità predittiva su dati complessi; Logistica è più interpretabile

Prompt